

Silo

Manual técnico

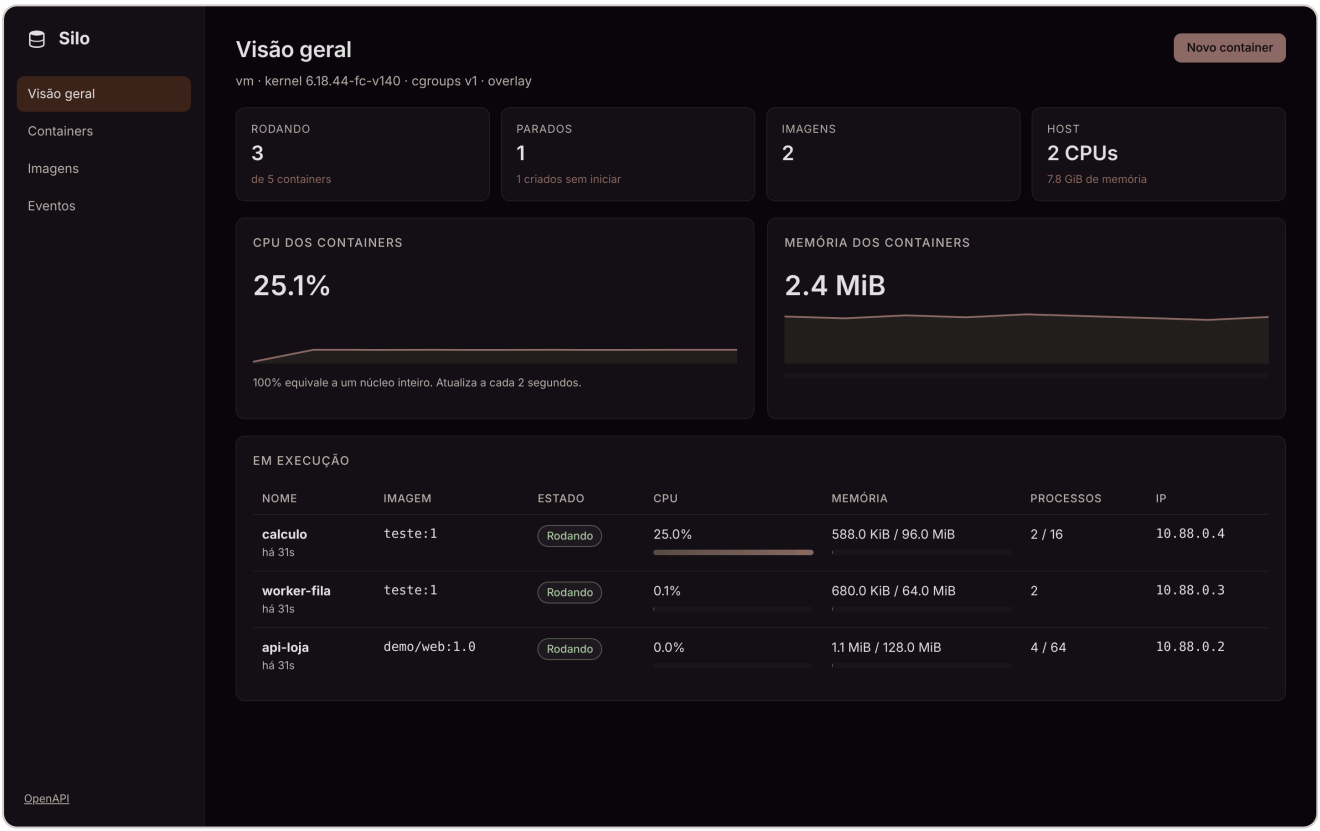
Runtime de containers em Go com namespaces, cgroups, pivot_root e overlayfs. Instalação, linha de comando, funcionamento interno, rede, imagens, painel web, API, configuração, extensão, testes e solução de problemas.

Copyright (©) 2026 Clara Portes

Versão 1.0.0 · Licença MIT

Sumário

01 Visão geral
02 Requisitos
03 Instalação
04 Primeiros passos
05 Linha de comando
06 Como o isolamento funciona
07 Limites com cgroups
08 Rede
09 Imagens e armazenamento
10 Painel web
11 API REST e WebSocket
12 Configuração
13 Como estender
14 Testes
15 Solução de problemas
16 Licença e créditos

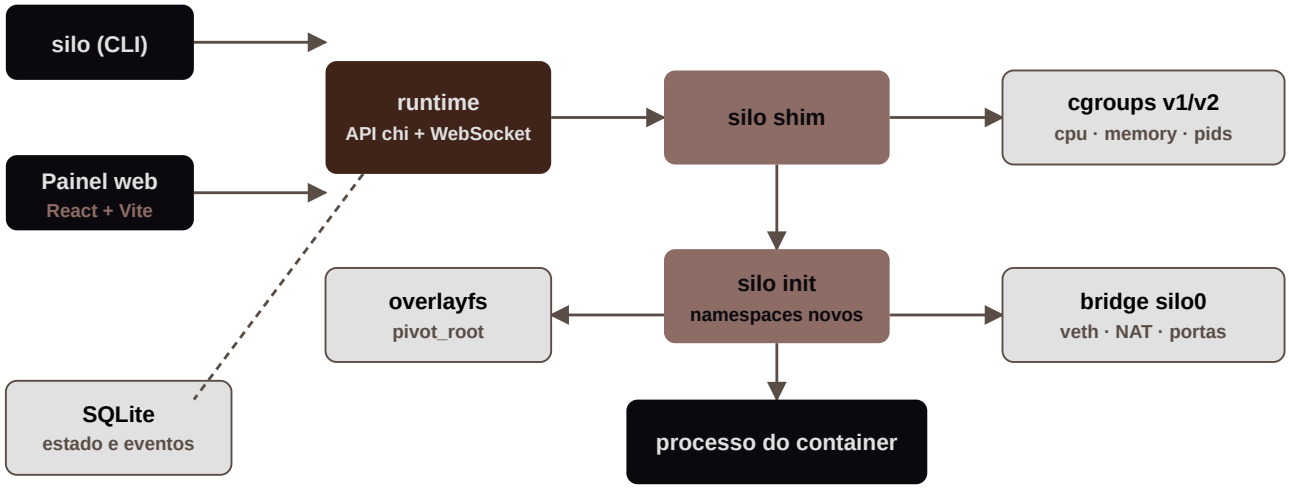


Painel do Silo com três containers rodando, cada um com seus limites de CPU, memória e processos.

1. Visão geral

O **Silo** é um runtime de containers simplificado, escrito em Go, que fala direto com o kernel Linux. Ele não depende de Docker, containerd ou runc: cria os namespaces com `clone()`, monta a raiz do container com overlayfs, troca a raiz com `pivot_root`, aplica limites com cgroups e liga a rede com pares veth numa bridge.

Tudo cabe em um único binário, `silo`, que serve ao mesmo tempo de linha de comando, supervisor de containers, processo inicial dentro do container e servidor da API com o painel web embutido.



Partes do sistema

Parte	Onde fica	O que faz
CLI	`cmd/silo`	Comandos do usuário e os pontos de entrada internos `shim`, `init` e `__enter`
Runtime	`internal/runtime`	Cria, inicia, para, supervisiona, executa comandos, lê saída, estatísticas e processos
Init	`internal/runtime/init.go`	Roda dentro dos namespaces novos: montagens, `pivot_root`, hostname, capabilities, usuário e `execve`
Cgroups	`internal/cgroups`	Mesma interface para cgroups v2 e v1: criar, limitar, adicionar processo, ler estatísticas e destruir
Rede	`internal/network`	Bridge, veth, NAT com iptables, alocação de IP, DNS e encaminhamento de portas TCP e UDP
Imagens	`internal/image`	Cliente de registro OCI/Docker com token, escolha de plataforma, verificação de digest, camadas e whiteouts
Banco	`internal/store`	SQLite em modo WAL com migrações versionadas

Parte	Onde fica	O que faz
API	`internal/api`	Rotas REST com chi, WebSocket, OpenAPI, métricas Prometheus e autenticação por token
Interface	`web/` e `internal/ui`	Painel em React + TypeScript compilado e embutido no binário com `go:embed`

Como um container sobe

1. `silo run` valida as opções, garante que a imagem existe (baixando se preciso) e grava o container no banco com estado **created**.
2. No modo `-d`, o Silo inicia `silo shim <id>` numa sessão própria (`setsid`). Sem `-d`, o próprio terminal faz o papel do shim.
3. O shim cria o cgroup com os limites, aloca um IP (com trava de arquivo para dois containers não pegarem o mesmo) e chama `silo init` com `CLONE_NEWPID | CLONE_NEWNS | CLONE_NEWUTS | CLONE_NEWIPC | CLONE_NEWNET`.
4. O init lê a configuração por um pipe e espera. O shim coloca o PID do init no cgroup, cria o par veth, move uma ponta para dentro do namespace de rede e configura IP, rota e loopback.
5. O shim libera o init, que entra num namespace de cgroup próprio, monta overlayfs, `/proc`, `/sys`, `/dev`, volumes e `/etc/hosts`, `/etc/hostname` e `/etc/resolv.conf`, faz o `pivot_root`, mascara caminhos sensíveis, define o hostname, remove capabilities, troca de usuário e chama `execve`.
6. Se o `execve` falhar, a mensagem volta por um terceiro pipe e o container fica **exited** com o erro. Se der certo, o pipe fecha sozinho (close-on-exec), o shim marca **running**, abre as portas publicadas e espera o processo terminar.
7. Quando o processo termina, o shim registra o código de saída, detecta OOM, fecha as portas, remove o veth e o cgroup e aplica a política de reinício.

2. Requisitos

Item	Versão mínima	Para quê
Linux	kernel 4.18 (5.x recomendado)	Namespaces, overlayfs e cgroups. No Windows use o WSL2, que já traz um kernel Linux completo
Usuário root	-	Criar namespaces, montar sistemas de arquivos, mexer em cgroups e interfaces de rede
Go	1.24	Compilar o binário
Compilador C	gcc ou clang	O driver do SQLite (go-sqlite3) usa cgo
iproute2	qualquer	Comando `ip` para bridge, veth, endereços e rotas
util-linux	2.35	Comando `nsenter` para `silo exec` e configuração da rede
iptables	qualquer	NAT dos containers para a internet (opcional: sem ele os containers só falam com o host)
Node.js	20 (22 recomendado)	Só para recompilar a interface. O repositório já traz a interface compilada

> O Silo detecta sozinho se o host usa cgroups v2 (Ubuntu 22.04+, Debian 11+, Fedora 31+, WSL2 recente) ou v1 (sistemas mais antigos) e se o kernel aceita overlayfs. Confira com `sudo silo info`.

3. Instalação

Linux

1. Instale as dependências do sistema. No Ubuntu ou Debian: `sudo apt install -y git golang build-essential iproute2 util-linux iptables`. No Fedora: `sudo dnf install -y git golang gcc iproute util-linux iptables`.
2. Confira se o Go é 1.24 ou mais novo com `go version`. Se for mais antigo, baixe em go.dev/dl.
3. Baixe o projeto: `git clone https://github.com/Violet-pixel/Container-Runtime-Engine.git` e entre na pasta.
4. Rode `./scripts/install.sh`. O script compila a interface (se houver Node.js), compila o binário e copia para `/usr/local/bin/silo`.
5. Teste com `sudo silo info`.

Windows com WSL2

1. Abra o PowerShell como administrador e rode `wsl --install -d Ubuntu`. Reinicie se o Windows pedir.
2. Abra o Ubuntu pelo menu Iniciar e crie o usuário do Linux.
3. Siga os passos da seção Linux dentro do Ubuntu.
4. O painel abre no navegador do Windows em `http://localhost:7070`, porque o WSL2 repassa o localhost.

> Se o seu WSL2 estiver em cgroups v1 e você quiser v2, crie `%UserProfile%\.wslconfig` com `[wsl2]` e `kernelCommandLine = cgroup_no_v1=all` e rode `wsl --shutdown`. O Silo funciona nos dois modos.

Serviço do systemd

Para deixar a API e o painel sempre no ar: `WITH_SERVICE=1 ./scripts/install.sh`. O serviço lê `/etc/silo/silo.env` (modelo em `deploy/silo.env.example`) e usa `KillMode=process`, então reiniciar o serviço não derruba os containers, que são supervisionados pelos seus próprios shims.

```
sudo mkdir -p /etc/silo
sudo cp deploy/silo.env.example /etc/silo/silo.env
sudo systemctl restart silo
journalctl -u silo -f
```

Compilação manual

```
cd web && npm ci && npm run build && cd ..   # opcional: interface
CGO_ENABLED=1 go build -trimpath -ldflags "-s -w" -o bin/silo ./cmd/silo
sudo install -m 0755 bin/silo /usr/local/bin/silo
```

Desinstalação

```
sudo silo ps -q | xargs -r sudo silo rm -f
sudo systemctl disable --now silo 2>/dev/null; sudo rm -f /etc/systemd/system/silo.service
sudo ip link del silo0
sudo rm -rf /var/lib/silo /usr/local/bin/silo
```

4. Primeiros passos

Baixar uma imagem e rodar

```
sudo silo pull alpine:3.20
sudo silo run --rm alpine:3.20 sh -c 'echo "olá de $(hostname)"; ps; ip addr'
```

O `ps` mostra só os processos do container, com o `sh` como PID 1, e o `ip addr` mostra uma interface `eth0` com IP na faixa 10.88.0.0/16.

Um servidor web com limites

```
sudo silo run -d --name web -p 8080:80 -m 128m --cpus 0.5 --pids-limit 64 nginx:alpine
curl http://localhost:8080
sudo silo ps
sudo silo stats web
```

Entrar no container

```
sudo silo exec -it web sh
sudo silo exec -e MODO=teste -w /etc web sh -c 'echo $MODO; pwd'
```

Saída, parada e remoção

```
sudo silo logs -n 50 web
sudo silo logs -f web          # Ctrl+C para sair
sudo silo stop web             # SIGTERM e, após 10 s, SIGKILL
sudo silo start web
sudo silo rm -f web
```

Sem internet ou sem registro

O script `scripts/mkrootfs.sh` monta um rootfs com `sh`, `ls`, `ps`, `ip` e outros programas copiados do próprio host, junto com as bibliotecas de que eles precisam.

```
./scripts/mkrootfs.sh base.tar.gz
sudo silo import base.tar.gz base:1 /bin/sh
sudo silo run --rm base:1 sh -c 'hostname; id; ls /'
```

Qualquer rootfs em `.tar` ou `.tar.gz` serve, por exemplo o "minirootfs" do Alpine ou um `docker export`.

5. Linha de comando

Todos os comandos precisam de root. Containers podem ser indicados pelo nome, pelo ID completo ou por um prefixo do ID com pelo menos 3 caracteres.

Comando	Opções	O que faz
`run IMAGEM [CMD...]`	todas as de `create` e `-d`	Cria e inicia. Sem `-d` fica no terminal, repassa sinais (Ctrl+C manda SIGTERM, o segundo manda SIGKILL) e devolve o código de saída do container
`create IMAGEM [CMD...]`	ver tabela abaixo	Cria sem iniciar e mostra o ID
`start CONTAINER...`	`-a` liga ao terminal	Inicia containers criados ou parados
`stop CONTAINER...`	`-t` segundos	SIGTERM, espera e SIGKILL. A política de reinício não age depois de um stop
`restart CONTAINER...`	`-t` segundos	Para e inicia de novo
`kill CONTAINER...`	`-s` sinal (nome ou número)	Envia um sinal ao processo principal. Padrão: KILL
`rm CONTAINER...`	`-f`	Remove. Com `-f` mata antes se estiver rodando
`prune`		Remove todos os containers parados ou só criados
`ps`	`-a`, `-q`, `--json`	Lista containers em execução (ou todos com `-a`)
`inspect ALVO...`		JSON com especificação, estado, caminhos em disco, cgroup e namespaces. Aceita imagens também
`logs CONTAINER`	`-f`, `-n` linhas	Saída padrão e de erro do container
`exec CONTAINER CMD...`	`-e`, `-w`, `-u uid:gid`	Executa dentro dos namespaces e do cgroup do container
`stats [CONTAINER...]`	`--no-stream`	CPU, memória, processos e rede, atualizando a cada segundo
`top CONTAINER`		Processos do cgroup com PID, PPID, UID, estado, memória e comando
`pull IMAGEM`		Baixa do registro. Ex.: `alpine`, `nginx:alpine`, `ghcr.io/org/app:1.0`
`import ARQ NOME [CMD...]`	`-` lê da entrada	Importa um rootfs em tar ou tar.gz

Comando	Opções	O que faz
`images` · `rmi IMAGEM...`	`-q`, `--json`	Lista e remove imagens. Uma imagem em uso não pode ser removida
`events`	`-f`, `-n`	Histórico de eventos e acompanhamento ao vivo
`info`	`--json`	Kernel, cgroups, armazenamento, rede, CPUs, memória e contadores
`serve`	`--addr`, `--token`	API REST e painel web

Opções de `run` e `create`

Opção	Exemplo	Efeito
`--name`	`--name api`	Nome único. Sem ele o Silo gera um, como `calmo_cedro`
`-m`	`-m 512m`, `-m 1g`	Limite de memória (mínimo 6 MB). A swap fica desligada para o container
`--cpus`	`--cpus 0.5`	Quantos núcleos o container pode usar no total
`--cpu-weight`	`--cpu-weight 200`	Prioridade relativa entre containers quando a CPU está disputada (padrão do kernel: 100)
`--pids-limit`	`--pids-limit 100`	Máximo de processos e threads; protege contra fork bomb
`-p`	`-p 8080:80`, `-p 127.0.0.1:53:53/udp`	Publica uma porta. Só com a rede bridge
`-v`	`-v /srv/dados:/dados:ro`	Monta uma pasta ou arquivo do host. Caminhos absolutos
`-e`	`-e TZ=America/Sao_Paulo`	Variável de ambiente; sobrescreve a da imagem
`--network`	`bridge` · `none` · `host`	Modo de rede (seção 8)
`--restart`	`on-failure:5`, `always`	Reinicia com espera crescente (100 ms até 12,8 s)
`--read-only`		Raiz somente leitura; use volumes para gravar
`-u`	`-u nobody`, `-u 1000:1000`	Usuário dentro do container (nome do `/etc/passwd` da imagem ou número)

Opção	Exemplo	Efeito
`-w`, `--hostname`, `--entrypoint`, `--dns`, `--label`		Diretório de trabalho, hostname, entrypoint, servidores DNS e rótulos
`--rm`		Remove o container quando ele termina
`--privileged`		Mantém todas as capabilities do root. Use só quando precisar

As opções vêm antes da imagem. Tudo o que vem depois da imagem é o comando: `silo run -m 64m alpine ls -la /`.

6. Como o isolamento funciona

Namespaces

Namespace	Flag	Efeito
PID	`CLONE_NEWPID`	O processo do container é o PID 1 e só enxerga os próprios filhos. Quando ele morre, o kernel encerra todos os processos do namespace
Montagem	`CLONE_NEWNS`	Montagens feitas dentro do container (overlayfs, /proc, volumes) não aparecem no host. O init marca tudo como privado antes de montar
UTS	`CLONE_NEWUTS`	Hostname próprio
IPC	`CLONE_NEWIPC`	Filas de mensagens, semáforos e memória compartilhada separados
Rede	`CLONE_NEWNET`	Interfaces, rotas, regras e portas próprias (exceto no modo host)
Cgroup	`unshare(CLONE_NEWCGROUP)`	O container vê o próprio cgroup como raiz em `/proc/self/cgroup`

Para ver os namespaces de um container: `sudo silo inspect web` (campo `namespaces`) ou `sudo ls -l /proc/<pid>/ns`.

Raiz do container

Com o armazenamento **overlay**, a pasta da imagem é a camada de baixo (somente leitura) e cada container tem uma pasta `upper` para o que ele escreve. Vários containers compartilham a mesma imagem sem copiar nada. Com o armazenamento **copy**, usado quando o kernel não aceita overlayfs, a imagem é copiada para a pasta do container na criação.

Depois de montar tudo, o init chama `pivot_root(nova_raiz, nova_raiz/.silo-old-root)`, entra na nova raiz e desmonta a antiga com `MNT_DETACH`. Assim nenhum caminho do host continua acessível. Se o `pivot_root` não for possível (por exemplo com a raiz em ramfs), o Silo usa `chroot`.

O que o container recebe em /dev e /etc

- `/dev` é um tmpfs novo com apenas `null`, `zero`, `full`, `random`, `urandom` e `tty`, mais `/dev/pts` próprio (`newinstance`) e `/dev/shm` de 64 MB.
- `/etc/hostname`, `/etc/hosts` e `/etc/resolv.conf` são gerados pelo Silo e montados por cima dos da imagem.
- `/proc` é novo (mostra só os processos do container) e `/sys` é montado somente leitura.

Segurança padrão

Proteção	Como é feita
Capabilities	Ficam só as 14 que o Docker também mantém (CHOWN, DAC_OVERRIDE, FOWNER, FSETID, KILL, SETGID, SETUID, SETPCAP, NET_BIND_SERVICE, NET_RAW, SYS_CHROOT, MKNOD, AUDIT_WRITE, SETFCAP). As demais, como SYS_ADMIN, SYS_MODULE e SYS_TIME, saem do bounding set antes do `execve`
Caminhos mascarados	`/proc/kcore`, `/proc/keys`, `/proc/timer_list` e `/proc/sched_debug` recebem `/dev/null`; `/proc/acpi` e `/sys/firmware` recebem um tmpfs vazio
Somente leitura	`/proc/sys`, `/proc/sysrq-trigger`, `/proc/irq` e `/proc/bus`
Usuário	`-u` troca para outro usuário com `setgroups`, `setgid` e `setuid` depois do `pivot_root`
Raiz	`--read-only` remonta a raiz como somente leitura
Extração de imagens	Arquivos com `..` ou que atravessariam um link simbólico para fora da raiz são recusados

> O Silo não usa namespaces de usuário: o root dentro do container é o root do host com menos capabilities. Para cargas que você não confia, combine com `-u`, `--read-only` e limites de recursos.

exec

`silo exec` chama o próprio binário como `silo __enter`, que primeiro se coloca no cgroup do container e depois executa `nsenter -t PID -m -u -i -p -C -n -r -w`. O comando entra em todos os namespaces, na raiz e no diretório do processo principal, e passa a contar nos limites do container.

7. Limites com cgroups

Cada container ganha um grupo `silo/<id>`. O Silo escreve os limites antes de liberar o processo e lê as estatísticas do mesmo grupo.

Opção	cgroups v2	cgroups v1
`--cpus 0.5`	`cpu.max = 50000 100000`	`cpu.cfs_quota_us = 50000`, `cpu.cfs_period_us = 100000`
`--cpu-weight 200`	`cpu.weight = 200`	`cpu.shares` convertido (2 a 262144)
`-m 256m`	`memory.max` e `memory.swap.max = 0`	`memory.limit_in_bytes` e `memory.memsw.limit_in_bytes`
`--pids-limit 64`	`pids.max = 64`	`pids.max = 64`

Estatística	cgroups v2	cgroups v1
Tempo de CPU	`cpu.stat usage_usec`	`cpuacct.usage`
Tempo contido pela cota	`cpu.stat throttled_usec`	`cpu.stat throttled_time`
Memória em uso e pico	`memory.current`, `memory.peak`	`memory.usage_in_bytes`, `memory.max_usage_in_bytes`
Mortes por falta de memória	`memory.events oom_kill`	`memory.oom_control oom_kill`
Processos	`pids.current`	`pids.current`

O percentual de CPU é calculado entre duas leituras: (tempo de CPU consumido / tempo decorrido) × 100. 100% equivale a um núcleo inteiro, então um container com `--cpus 0.5` fica perto de 50% mesmo rodando um laço infinito.

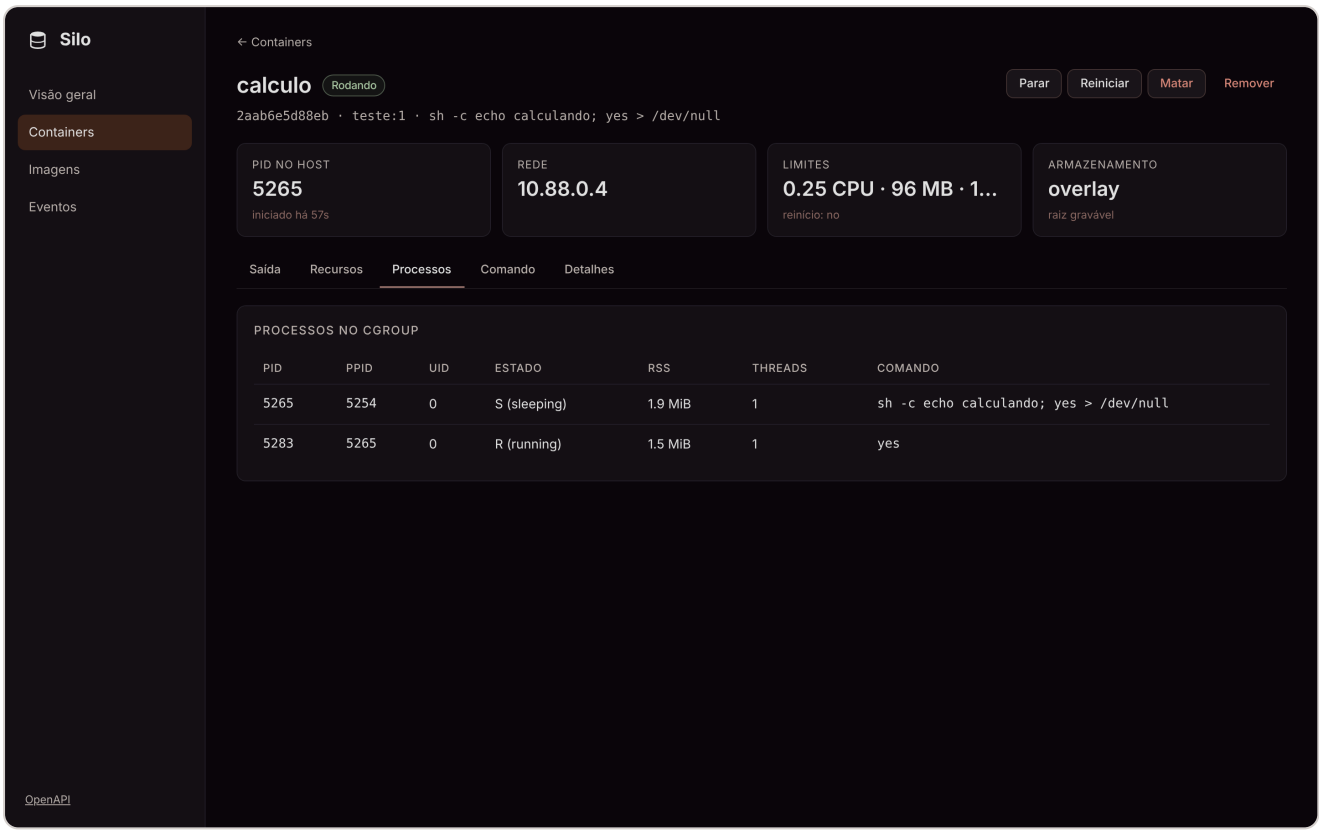
Testando os limites

```
sudo silo run -d --name cpu --cpus 0.25 alpine sh -c 'yes > /dev/null'
sudo silo stats --no-stream cpu               # perto de 25%

sudo silo run --rm -m 16m alpine sh -c 'dd if=/dev/zero of=/dev/shm/x bs=1M count=64'
# "Killed": o kernel encerrou o dd; o inspect mostra o erro de OOM

sudo silo run --rm --pids-limit 8 alpine sh -c 'for i in $(seq 20); do sleep 5 & done; wait'
# "can't fork": o nono processo é recusado
```

Quando há OOM, o container recebe o erro "o processo foi encerrado por falta de memória (OOM)" e o evento `oom`.



Processos dentro do cgroup de um container limitado a 16 processos.

8. Rede

Modo bridge (padrão)

1. Na primeira vez, o Silo cria a bridge `silo0` com o IP `10.88.0.1/16`, liga o encaminhamento de pacotes e adiciona as regras `MASQUERADE` (saída para a internet) e `FORWARD` no iptables.
2. Para cada container, aloca o menor IP livre da faixa, cria o par veth `vs<id>` / `vc<id>`, liga a ponta do host na bridge e move a outra para o namespace do container, onde ela vira `eth0`.
3. Dentro do container configura IP, loopback e rota padrão para `10.88.0.1`.
4. Quando o container para, o veth é apagado e o IP volta a ficar livre.

Portas publicadas

Cada `-p HOST:CONTAINER` abre um proxy em Go no shim do container, que escuta no host e repassa para `IP_DO_CONTAINER:PORTA`. Funciona para acessos de fora e também de `localhost`, em TCP e UDP. O Silo recusa iniciar um container se outra instância em execução já usa a mesma porta.

```
sudo silo run -d -p 8080:80 nginx:alpine               # todas as interfaces
sudo silo run -d -p 127.0.0.1:5432:5432 postgres:16     # só no host local
sudo silo run -d -p 5353:53/udp alpine nc -lu -p 53     # UDP
```

Modo none

`--network none` cria um namespace de rede só com a loopback. Útil para tarefas que não devem falar com nada.

Modo host

`--network host` não cria namespace de rede: o container usa as interfaces do servidor. Não aceita `-p`, porque as portas já são as do host.

DNS

O `/etc/resolv.conf` do container usa, nesta ordem, os servidores de `--dns`, os do host (ignorando endereços de loopback como o `127.0.0.53` do systemd-resolved) ou `1.1.1.1` e `8.8.8.8`.

Faixa própria

```
SILO_BRIDGE=silo1 SILO_BRIDGE_CIDR=172.30.0.1/24 sudo -E silo run ...
```

9. Imagens e armazenamento

Baixando de registros

O cliente OCI do Silo segue o fluxo da Distribution API: pede o manifesto, recebe 401 com o desafio `WWW-Authenticate: Bearer`, busca o token no serviço indicado e repete. Se o manifesto for uma lista de plataformas, escolhe `linux/<arquitetura do host>` (ou `SILO_ARCH`). Depois baixa a configuração e cada camada, confere o SHA-256 de cada uma e aplica na ordem.

Referência	Vira
`alpine`	`registry-1.docker.io/library/alpine:latest`
`nginx:alpine`	`registry-1.docker.io/library/nginx:alpine`
`ghcr.io/org/app:1.0`	`ghcr.io/org/app:1.0`
`localhost:5000/teste`	registro local em HTTP
`busybox@sha256:...`	versão exata pelo digest

Registro privado: defina `SILO_REGISTRY_USER` e `SILO_REGISTRY_PASSWORD`. Registro sem HTTPS: inclua em `SILO_INSECURE_REGISTRIES`. Camadas zstd ainda não são suportadas.

Camadas e whiteouts

Cada camada é um tar (normalmente gzip). Um arquivo `.wh.nome` apaga `nome` das camadas anteriores e `.wh..wh..opq` esvazia o diretório. Links, links simbólicos, permissões, donos, setuid e datas são preservados.

Importando

```
sudo silo import rootfs.tar.gz minha-base:1 /bin/sh
cat rootfs.tar | sudo silo import - outra:2 /usr/bin/app --porta 80
```

Organização em disco

```
/var/lib/silo/
  silo.db, silo.db-wal       banco de estado (ver o PDF do banco de dados)
  network.lock               trava da alocação de IPs
  images/<id>/rootfs/        sistema de arquivos de cada imagem
  containers/<id>/
    upper/ work/ rootfs/     overlayfs (ou rootfs copiado no modo copy)
    container.log            saída do processo
    shim.log                 mensagens do supervisor
    hostname hosts resolv.conf
  tmp/                       downloads em andamento
```

Saída dos containers

A saída padrão e a de erro vão direto para `container.log`, sem passar pelo supervisor, então nada se perde se ele cair. O arquivo cresce enquanto o container existir; remova o container ou esvazie com `sudo truncate -s 0 /var/lib/silo/containers/<id>/container.log`.

10. Painel web

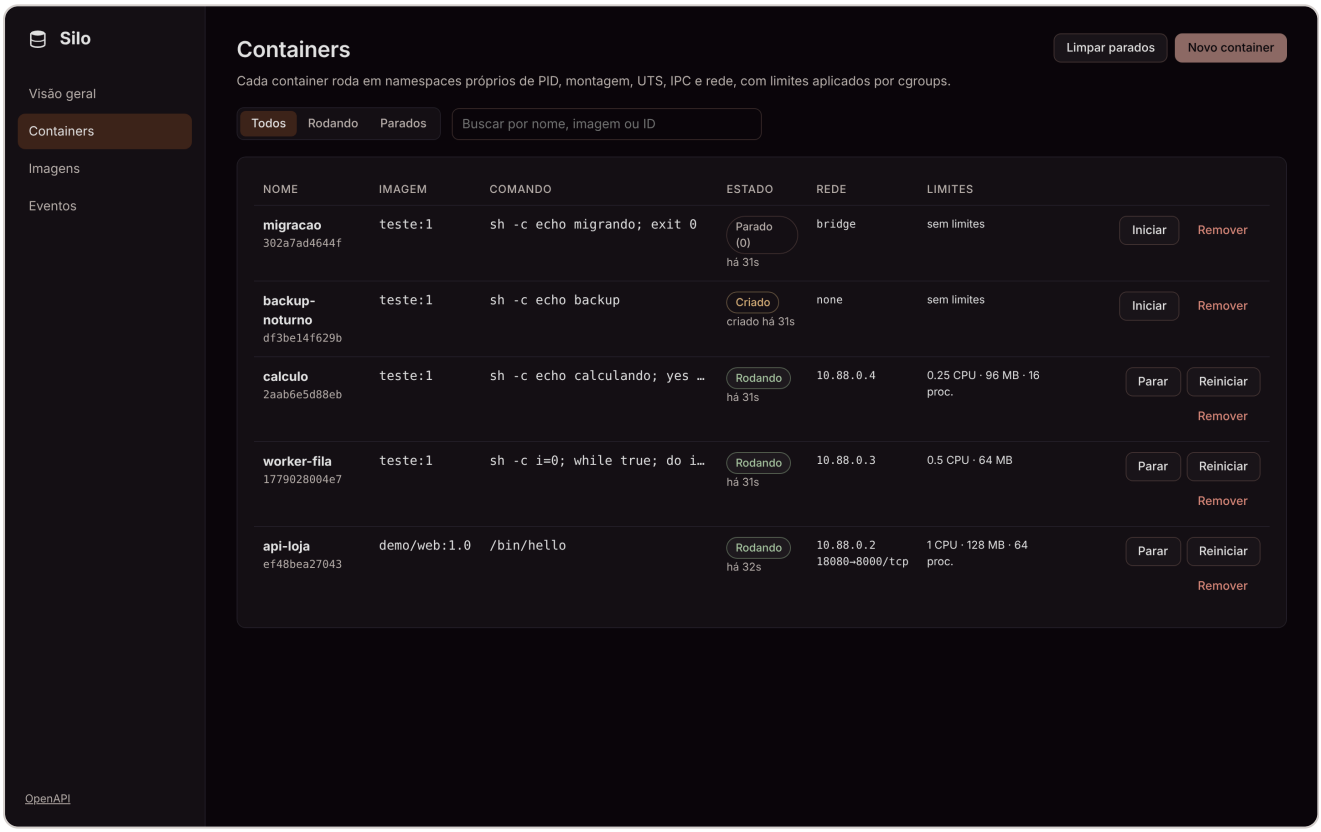
Rode `sudo silo serve` e abra `http://127.0.0.1:7070`. O painel é servido pelo próprio binário e mostra também o que for feito pela linha de comando.

Visão geral

Contadores de containers e imagens, gráficos de CPU e memória somados e a tabela dos containers rodando com o uso de cada um frente ao limite.

Containers

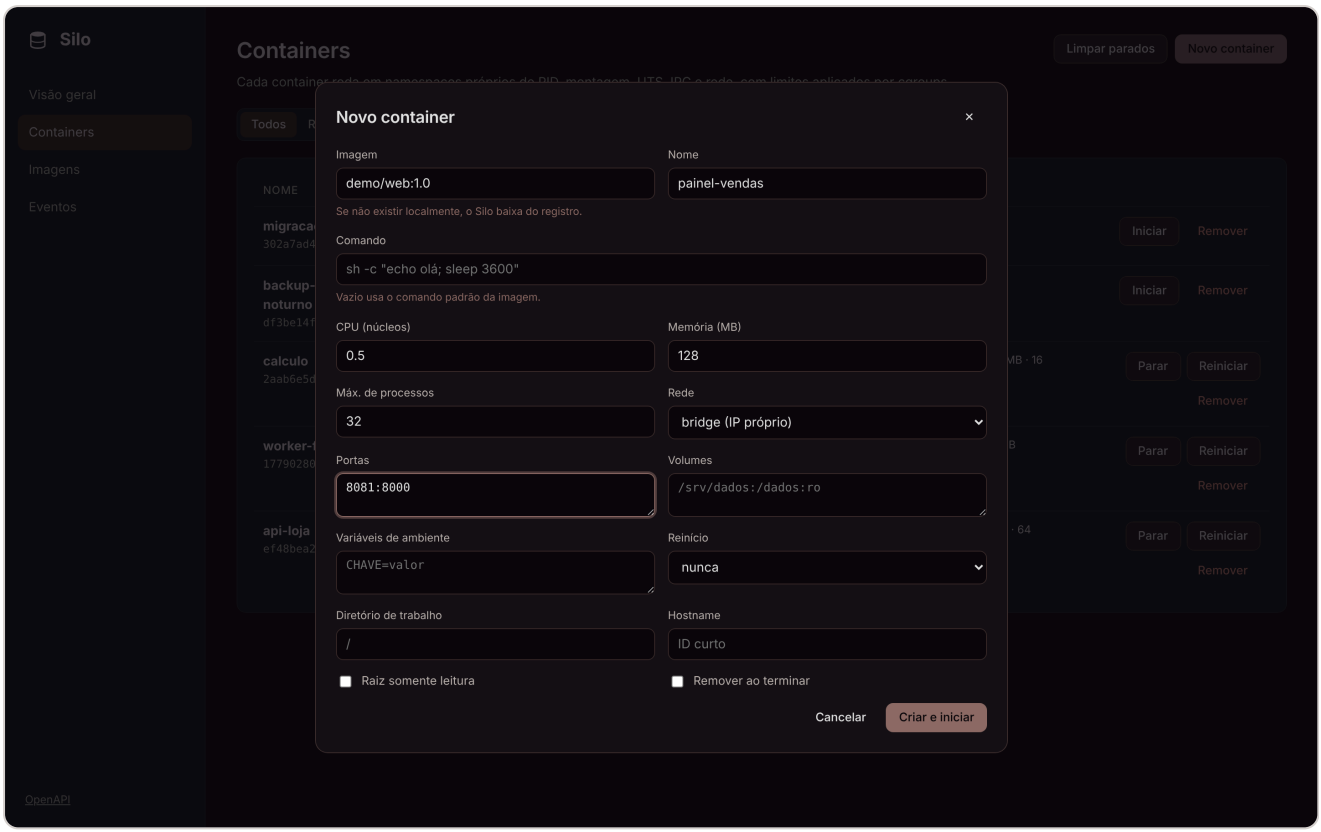
Lista com filtro (todos, rodando, parados), busca, estado, rede, portas e limites, com ações de iniciar, parar, reiniciar e remover. "Limpar parados" remove de uma vez os que não estão rodando.



Lista de containers.

Novo container

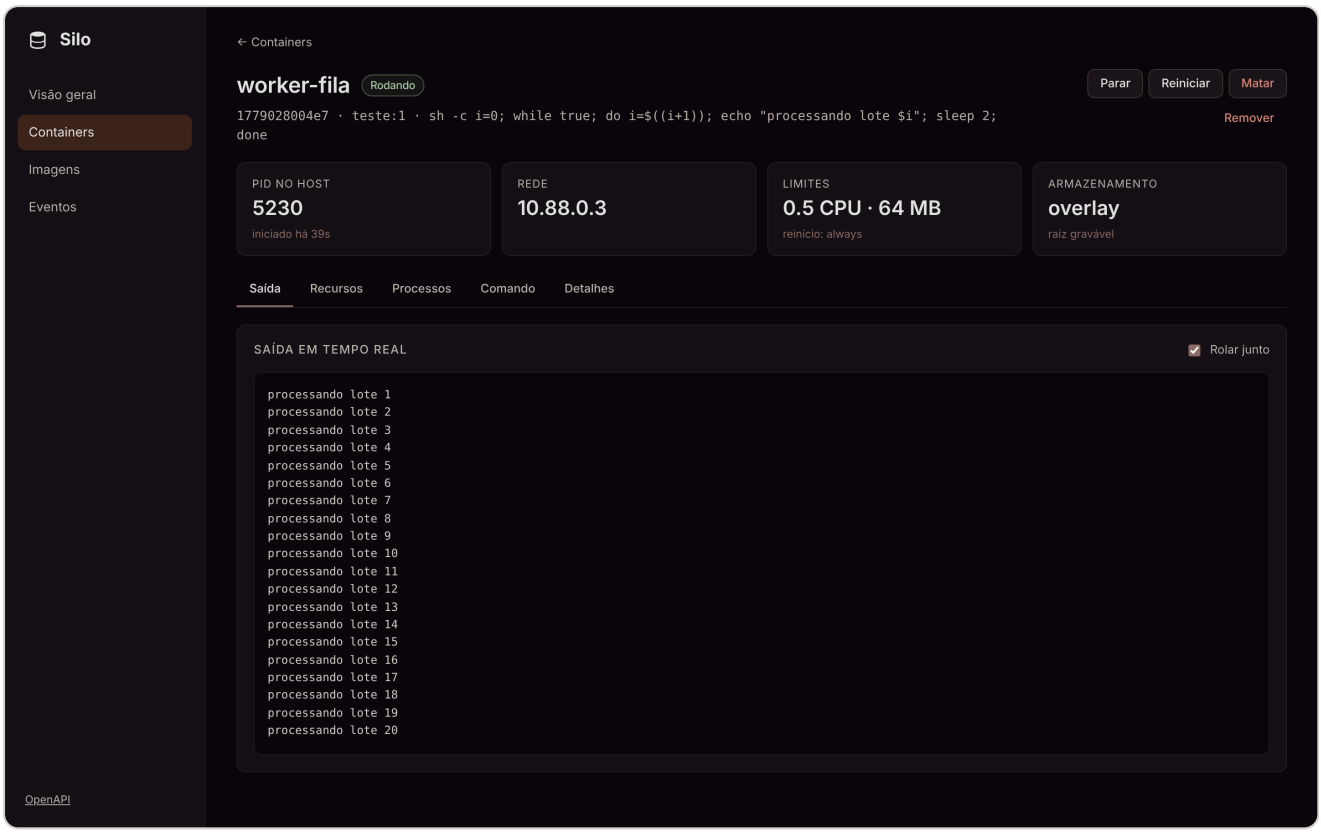
Formulário com imagem (com sugestões das locais; se não existir, o Silo baixa), nome, comando, CPU, memória, processos, rede, portas, volumes, variáveis, reinício, diretório, hostname, raiz somente leitura e remoção automática.



Criação de um container com limites e porta publicada.

Detalhe do container

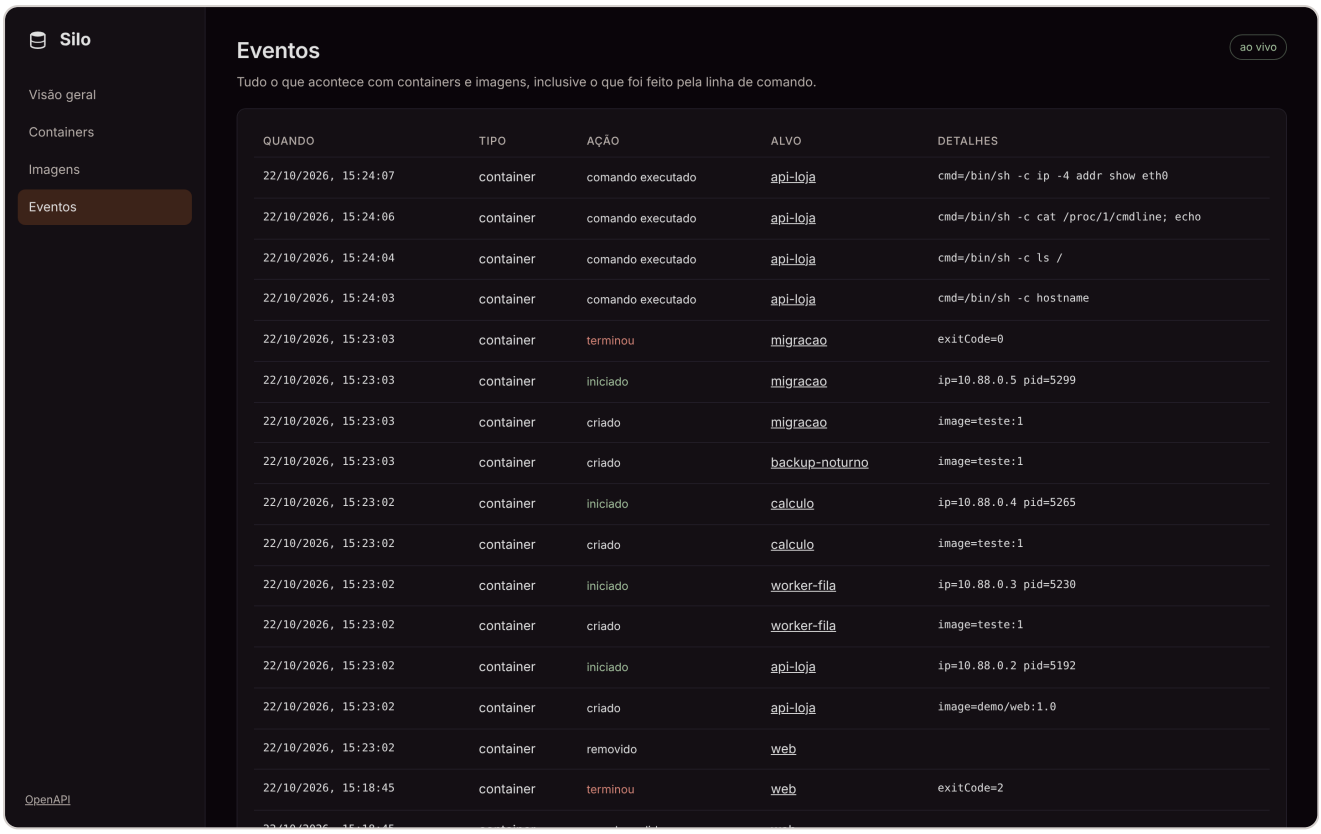
- **Saída:** a saída em tempo real por WebSocket, com rolagem automática.
- **Recursos:** CPU, memória, processos e rede atualizados a cada 1,5 segundo, com o limite de cada um.
- **Processos:** o `top` do cgroup.
- **Comando:** executa comandos curtos com `/bin/sh -c` dentro do container (até 60 segundos e 1 MB de saída). Para sessões interativas use `silo exec -it`.
- **Detalhes:** os namespaces e o JSON completo do `inspect`.



Saída de um container em tempo real.

Imagens e eventos

Em Imagens dá para baixar com barra de progresso, importar um tar e remover. Em Eventos aparece cada ação (criado, iniciado, terminou com código, OOM, reiniciado, removido, imagem baixada) ao vivo.



Eventos ao vivo.

Token e acesso remoto

Com `--token`, o painel pede o token na primeira vez e guarda no navegador. Para abrir de outra máquina use `--addr 0.0.0.0:7070 --token ...`; sem token o Silo recusa escutar fora do localhost. Para HTTPS, coloque um proxy reverso (Nginx, Caddy) na frente.

11. API REST e WebSocket

Base: `http://127.0.0.1:7070/api/v1`. A especificação completa fica em `/api/openapi.yaml` e pode ser aberta no Swagger Editor ou importada no Postman e no Insomnia.

Método e rota	Descrição
`GET /info`	Versão, kernel, versão dos cgroups, armazenamento, rede, CPUs, memória e contadores
`GET /containers?all=true`	Lista containers
`POST /containers`	Cria. Corpo com os campos do OpenAPI; `start` (padrão true) e `pull` (padrão true)
`GET /containers/{{id}}`	Inspeção completa
`DELETE /containers/{{id}}?force=true`	Remove
`POST /containers/{{id}}/start`	Inicia
`POST /containers/{{id}}/stop?t=10`	Para
`POST /containers/{{id}}/restart`	Reinicia
`POST /containers/{{id}}/kill?signal=15`	Envia sinal (número)
`POST /containers/{{id}}/exec`	`{{"cmd": ["ls", "/"]}}` devolve `exitCode` e `output`
`GET /containers/{{id}}/logs?tail=500`	Texto
`GET /containers/{{id}}/logs/stream`	WebSocket: cada mensagem é um trecho da saída
`GET /containers/{{id}}/stats` · `/top`	Recursos e processos
`GET /stats`	Recursos de todos os containers rodando
`POST /containers/prune`	Remove os parados
`GET /images` · `GET /images/{{id}}` · `DELETE /images/{{id}}`	Imagens
`POST /images/pull`	`{{"reference": "alpine:3.20"}}`; resposta NDJSON: `stage` / `current` / `total` e por fim `done` ou `error`
`POST /images/import?reference=x:1&cmd=/bin/sh`	Corpo: o tar
`GET /events?after=0&limit=200`	Histórico
`GET /events/stream`	WebSocket com cada evento novo em JSON
`GET /metrics`	Formato Prometheus

Erros

Status	code	Quando
400	invalid	Parâmetro inválido, JSON com campo desconhecido, identificador ambíguo
401	unauthorized	Token ausente ou errado
404	not_found	Container ou imagem inexistente
409	conflict	Nome repetido, container já rodando ou parado, imagem em uso, porta ocupada
500	internal	Falha do sistema (mensagem detalhada no corpo)

Exemplos

```
TOKEN=um-token-longo
curl -s -H "Authorization: Bearer $TOKEN" http://127.0.0.1:7070/api/v1/containers

curl -s -X POST http://127.0.0.1:7070/api/v1/containers -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{
    "name": "fila", "image": "alpine:3.20", "cmd": ["sh", "-c", "while true; do date; sleep
5; done"],
    "resources": {"cpus": 0.25, "memoryMb": 64, "pidsLimit": 32},
    "restart": {"name": "always"}
  }'

curl -s -X POST -H "Authorization: Bearer $TOKEN"
http://127.0.0.1:7070/api/v1/containers/fila/stop
websocat "ws://127.0.0.1:7070/api/v1/events/stream?token=$TOKEN"
```

Prometheus

```
scrape_configs:
  - job_name: silo
    metrics_path: /api/v1/metrics
    bearer_token: um-token-longo
    static_configs: [{ targets: ['127.0.0.1:7070'] }]
```

Métricas: `silo_containers{state}`, `silo_images`, `silo_container_cpu_seconds_total`, `silo_container_memory_bytes` e `silo_container_pids`.

12. Configuração

Tudo é configurado por variáveis de ambiente. Com `sudo`, use `sudo -E` ou `sudo VAR=valor silo ...`. No serviço, edite `/etc/silo/silo.env`.

Variável	Padrão	Descrição
`SILO_ROOT`	`/var/lib/silo`	Pasta de dados. Use o mesmo valor na CLI e no serviço
`SILO_ADDR`	`127.0.0.1:7070`	Endereço do `silo serve`
`SILO_TOKEN`	vazio	Token da API. Obrigatório fora do localhost
`SILO_ALLOW_INSECURE`	vazio	Com `1`, permite escutar fora do localhost sem token (não recomendado)
`SILO_BRIDGE`	`silo0`	Nome da bridge
`SILO_BRIDGE_CIDR`	`10.88.0.1/16`	Gateway e sub-rede
`SILO_STORAGE`	`auto`	`overlay`, `copy` ou `auto`
`SILO_STOP_TIMEOUT`	`10`	Segundos entre SIGTERM e SIGKILL
`SILO_EVENTS_RETAIN`	`5000`	Eventos guardados no banco
`SILO_REGISTRY_USER`, `SILO_REGISTRY_PASSWORD`	vazio	Credenciais para registros privados
`SILO_INSECURE_REGISTRIES`	vazio	Registros em HTTP, separados por vírgula
`SILO_ARCH`	do host	Arquitetura a escolher nas listas de manifestos
`SILO_CORS_ORIGIN`	vazio	Origem liberada para uma interface hospedada em outro endereço

13. Como estender

Novo comando da CLI

1. Em `cmd/silo/main.go`, acrescente uma entrada na lista `commands`.
2. Escreva a função usando `newFlags` para as opções e os métodos de `runtime.Runtime`.

```
{"pause", "CONTAINER", "Congela os processos", cmdPause},

func cmdPause(rt *runtime.Runtime, args []string) error {
        c, err := rt.Get(args[0])
        if err != nil {
                return err
        }
        return os.WriteFile(filepath.Join("/sys/fs/cgroup/silo", c.ID, "cgroup.freeze"),
[]byte("1"), 0o644)
}
```

Nova rota da API

1. Registre em `Router()` (`internal/api/server.go`), dentro de `r.Route("/{id}", ...)`.
2. Implemente o handler em `handlers.go` usando `writeJSON` e `s.fail`, que já converte os erros do runtime em 400, 404 ou 409.
3. Documente em `internal/api/openapi.yaml` e, se quiser, crie a função em `web/src/api.ts`.

```
r.Post("/pause", s.pause)

func (s *Server) pause(w http.ResponseWriter, r *http.Request) {
        if err := s.rt.Pause(chi.URLParam(r, "id")); err != nil {
                s.fail(w, err)
                return
        }
        writeJSON(w, 200, map[string]bool{"paused": true})
}
```

Novo limite de cgroup

1. Acrescente o campo em `store.Resources` (vai junto no JSON da especificação, sem migração de banco).
2. Acrescente em `cgroups.Limits` e passe o valor em `runOnce` (`supervise.go`).
3. Escreva o arquivo nas duas versões: em `v2.Update` (por exemplo `io.max`) e em `v1.Update` (`blkio.throttle.*`), adicionando o controlador em `enableControllers` e em `v1Controllers`.
4. Crie a opção na CLI em `bindSpec` e o campo no formulário do painel.

Novo passo dentro do container

`setupContainer` em `internal/runtime/init.go` roda nos namespaces novos e antes do `execve`. É o lugar para rlimits, filtros seccomp, montagens extras ou variáveis calculadas. Antes do `pivot`, os caminhos são do host; depois dele, do container.

```
var lim syscall.Rlimit = syscall.Rlimit{Cur: 1024, Max: 1024}
if err := syscall.Setrlimit(syscall.RLIMIT_NOFILE, &lim); err != nil {
        return fmt.Errorf("rlimit: %w", err)
}
```

Novo modo de rede ou armazenamento

Modos de rede são tratados em `runOnce` (criação) e validados em `network.ValidMode`. Drivers de armazenamento são escolhidos em `StorageDriver` (`runtime.go`), preparados em `Create` e montados em `setupContainer`.

Interface

O código fica em `web/src`: `api.ts` (tipos e chamadas), `components/ui.tsx` (peças visuais), `pages/` (telas) e `styles.css` (cores da paleta em variáveis CSS no topo). Rode `sudo silo serve` num terminal e `make dev` em outro para editar com recarga automática; `make ui` gera a versão final embutida no binário.

14. Testes

Comando	O que cobre
`make test`	Unidade e API: leitura de portas, volumes e memória, validação, mesclagem de variáveis, camadas com whiteouts, links e tentativas de escapar da raiz, referências de imagem, desafio de autenticação, escolha de plataforma, banco (conflitos, portas, IPs, prefixos, eventos), alocação de IPs, DNS, autenticação da API, códigos de erro, validação de criação, interface e OpenAPI
`make e2e`	Como root, com um rootfs gerado: PID 1 e hostname isolados, código de saída repassado, rede none, limites de memória, CPU e processos aplicados de verdade, exec bloqueado pelo pids-limit, stop, rm, conflito de porta e a API subindo
`make lint`	`go vet` e checagem de tipos da interface

O workflow de CI em `.github/workflows/ci.yml` compila a interface, roda `gofmt`, `go vet`, os testes com `-race`, compila o binário e roda os testes ponta a ponta num runner Ubuntu com cgroups v2. O binário fica disponível como artefato de cada execução.

15. Solução de problemas

Sintoma	Causa provável	Solução
`o Silo precisa rodar como root`	Comando sem sudo	Use `sudo silo ...`
`mount overlay ... invalid argument`	Kernel ou sistema de arquivos sem overlayfs (por exemplo a pasta de dados em outro overlay)	`SILO_STORAGE=copy` ou mova `SILO_ROOT` para ext4/xfs
`o comando "ip" não foi encontrado`	iproute2 ausente	`sudo apt install iproute2 util-linux`
Container sem internet	Falta iptables, encaminhamento desligado ou firewall bloqueando a bridge	Instale o iptables, confira `sysctl net.ipv4.ip_forward` e libere `silo0` no firewall (ufw: `sudo ufw allow in on silo0`)
`comando "x" não encontrado no container`	A imagem não tem o programa ou o PATH é diferente	Use o caminho completo ou confira com `silo run --rm IMAGEM ls /bin`
`a porta 8080/tcp já está em uso`	Outro container rodando usa a porta	Pare o outro ou escolha outra porta. Se for um programa do host, o erro aparece no `silo logs`
`limite de memória: ... permission denied`	Controlador não habilitado (cgroups v2 delegado) ou sistema sem cgroups	Rode como root no host (não dentro de outro container sem privilégios). Confira `cat /sys/fs/cgroup/cgroup.controllers`
Pull falha com 429	Limite de downloads anônimos do Docker Hub	Defina `SILO_REGISTRY_USER` e `SILO_REGISTRY_PASSWORD` ou espere
`camadas zstd não são suportadas`	Imagem publicada com compressão zstd	Use outra tag ou importe com `docker export` e `silo import`
Container aparece como saiu (255) depois de reiniciar a máquina	Os processos não sobrevivem ao desligamento	É o comportamento esperado; inicie com `silo start` ou use `--restart always` com o serviço
Painel pede token sem parar	Token diferente do servidor	Clique em Sair, confira `SILO_TOKEN` e entre de novo

Onde olhar

- `sudo silo inspect NOME` : campos `error` , `exitCode` e caminhos.
- `/var/lib/silo/containers/<id>/shim.log` : mensagens do supervisor.

- `sudo silo events -n 100`: sequência de tudo o que aconteceu.
- `journalctl -u silo`: log da API quando instalada como serviço.

16. Licença e créditos

Copyright (©) 2026 Clara A. Portes. Distribuído sob a licença MIT: você pode usar, copiar, modificar e redistribuir o código, inclusive para fins comerciais, desde que mantenha o aviso de copyright e o texto da licença.

Componentes de terceiros

Componente	Uso	Licença
chi	Roteador HTTP	MIT
gorilla/websocket	WebSocket	BSD-2-Clause
go-sqlite3 (com SQLite)	Banco de estado	MIT (SQLite em domínio público)
React e Vite	Interface	MIT
TypeScript	Tipagem da interface	Apache-2.0

`ip`, `nsenter` e `iptables` são programas do sistema chamados pelo Silo e não fazem parte deste repositório.